# TRUCO YA

El truco uruguayo dentro de Minecraft — Informe y documentación técnica

Programación Avanzada — UTU
**Grupo 1**
Nicolás Bentancour · Agustín Perdomo · Matías Piñeyro · Thiago Rodríguez

Versión 1.7.0 — 21/10/2026
Repositorio: github.com/Nicrolax/Grupo1_proyecto

# Índice


1. 1. Desarrollo teórico de la situación problema
2. 2. Requisitos
3. 3. Stack tecnológico
4. 4. Diagramas UML
5. 5. Casos de uso
6. 6. Diagrama de infraestructura
7. 7. Pruebas
8. 8. Calidad del código y análisis estático

# 1. Desarrollo teórico de la situación problema

**Proyecto:** Truco Ya, el truco uruguayo dentro de Minecraft **Curso:** Programación Avanzada (UTU)
**Grupo 1:** Nicolás Bentancour, Agustín Perdomo, Matías Piñeyro y Thiago Rodríguez

---

## 1.1 Contexto

El truco es el juego de cartas más jugado del Uruguay. Se juega en familia, en el liceo, en los clubes y en las ruedas de mate. La versión uruguaya tiene reglas propias que la distinguen de la argentina:

- la **muestra**;
- las **piezas** (el 2, 4, 5, 10 y 11 del palo de la muestra);
- la **flor** con contraflor y "con flor me achico";
- el partido a 30 o 40 puntos, partido en **malas y buenas**.

Esas reglas se aprenden jugando con alguien que ya sabe, y cada vez menos jóvenes las aprenden.

Al mismo tiempo, los adolescentes pasan muchas horas en Minecraft, uno de los juegos más jugados del mundo. Minecraft no tiene ningún juego de cartas. Los mods de cartas que existen copian juegos genéricos (póker, blackjack) o la versión española del truco, y no respetan las reglas uruguayas.

## 1.2 El problema

> **No existe una forma de jugar al truco uruguayo, con sus reglas reales, dentro del mundo de Minecraft: ni contra la computadora ni con amigos en un servidor. Tampoco hay una que enseñe el juego y su cultura (el mate, la pulpería, el gaucho) a quien no lo conoce.**

Consecuencias:

- Quien quiere jugar al truco con sus amigos en Minecraft no puede. Tiene que salir del juego y usar otra aplicación, que tampoco suele tener las reglas uruguayas.
- Quien no sabe jugar no tiene dónde practicar a su ritmo, con un rival que respete las reglas y le muestre qué puede cantar en cada momento.
- La cultura rural uruguaya (pulperías, gauchos, payadas, mate) casi no aparece en los videojuegos que juegan los adolescentes.

## 1.3 Actores

| Actor | Quién es | Qué necesita del sistema |
|---|---|---|
| **Jugador** | Persona que juega Minecraft Java 1.16.5 con el mod instalado, sola o en un servidor. | Jugar partidos completos de truco uruguayo, contra paisanos (NPC) o contra otros jugadores, con las reglas correctas y una interfaz clara. |
| **Grupo de amigos (jugadores en red)** | Dos a cuatro jugadores conectados al mismo servidor. | Sentarse a la misma mesa (1 contra 1 o 2 contra 2), ver solo sus propias cartas, hacerse señas con el compañero. |
| **Administrador del servidor** | Quien levanta un servidor dedicado para su grupo. | Instalar el mod fácilmente y configurar qué puede afectar a todos (hora, clima, pedidos). Que nadie pueda hacer trampa desde su cliente. |
| **Paisano (NPC, actor del sistema)** | Personajes controlados por la inteligencia artificial del mod. | Jugar con distintos niveles de habilidad, aceptar invitaciones, ofrecer pedidos. |
| **Docente / evaluador** | Quien prueba el producto en vivo. | Instalar, ejecutar y verificar cada requisito en poco tiempo, con una guía de prueba. |

## 1.4 Alcance

### Dentro del alcance

1. Un motor completo del truco uruguayo: - muestra y piezas; - envido, real envido y falta envido; - flor, contraflor, contraflor al resto y con flor me achico; - truco, retruco y vale cuatro; - me voy al mazo, pardas, malas y buenas, partidos a 30 o 40; - 1 contra 1 y 2 contra 2.
2. Objetos del juego para jugar en el mundo: la Mesa de Truco, las Sillas Criollas y un Anotador. Además, un tapete (interfaz gráfica) que muestra solo las acciones legales.
3. Paisanos con inteligencia artificial de cinco niveles, que leen la mesa y "mienten" (farolean).
4. Progresión y modo historia: - Prestigio Criollo; - seis regiones con pulperías generadas en el mundo; - torneos, sellos y la Estancia del Caudillo; - el Viejo Vizcacha, cinemáticas y el capítulo de La Salamanca.
5. Contenido de la cultura local: - mate, pava y yerba; - picada y bebidas; - discos de música criolla; - carreras cuadreras; - armas gauchas.
6. Barajas coleccionables, pedidos de los paisanos, desafíos del día y estadísticas en la Libreta.
7. Juego en red sobre un servidor dedicado, con el servidor como única autoridad.
8. Modo tráiler, para grabar un video promocional.

### Fuera del alcance

- **Apuestas con dinero real** o cualquier moneda fuera del juego. Las apuestas usan solo esmeraldas de Minecraft.
- **Otras variantes del truco**: argentino sin flor, truco español o gallo. El mod implementa solo el uruguayo, con la flor desactivable.
- **Otras versiones de Minecraft**: Bedrock (consolas y celulares) y Java distinta de 1.16.5.
- **Rankings globales en internet, cuentas propias o base de datos externa.** Los datos quedan en cada mundo y en cada servidor.
- **Moderación de chat o anti-trampas** más allá de validar en el servidor cada jugada.

## 1.5 Alternativas consideradas

| Alternativa | Ventajas | Por qué se descartó |
|---|---|---|
| **A. Aplicación web o móvil de truco** (por ejemplo, React y Node.js) | Fácil de distribuir; se juega en el celular. | No resuelve el problema planteado: el público ya está dentro de Minecraft y tendría que salir del juego. Además, ya existen apps de truco; la novedad de este proyecto es llevarlo al mundo donde están los jugadores. |
| **B. Plugin de servidor (Spigot/Paper)** | No requiere instalar nada en el cliente. | Un plugin no puede agregar modelos 3D, texturas propias, animaciones, sonidos ni pantallas nuevas sin "trucos" con ítems de vanilla. No permitiría la mesa con cartas visibles, los gauchos animados ni el tapete. |
| **C. Datapack de Minecraft** | Sin programar en Java. | La lógica del truco (estados de la mano, envido, flor, IA con simulación) es inviable con comandos. Tampoco permite interfaces nuevas. |
| **D. Mod para Fabric** | Más liviano y actualizado. | En 1.16.5, Forge trae de fábrica lo que el proyecto necesita: canal de red entre cliente y servidor, configuración de servidor, generación de estructuras, eventos y registro de contenido. Con Fabric habría que sumar varias librerías aparte. Además, es el cargador con más mods compatibles en esa versión. |
| **E. Minecraft más nuevo (1.20+)** | Más funciones del motor. | 1.16.5 sigue siendo de las versiones con más mods disponibles, corre en computadoras modestas y usa Java 8, que viene con el entorno de desarrollo del proyecto ( `scripts\probar.bat` lo descarga solo). |
| **F. Mod para Forge 1.16.5 (elegida)** | Contenido propio completo (bloques, entidades, pantallas, sonidos, estructuras), cliente y servidor con la misma base de código, y servidor dedicado para jugar en grupo. | — |

## 1.6 Por qué la solución elegida resuelve el problema

- **Lleva el juego a donde ya está el público:** se juega al truco sin salir de Minecraft, solo o con amigos.
- **Respeta las reglas uruguayas:** un motor propio, separado de Minecraft ( `com.trucoya.core` ), implementa las reglas y se prueba de forma automática (33 pruebas).
- **Enseña jugando:** el tapete muestra solo las jugadas legales, el historial explica cada mano y los paisanos de nivel bajo perdonan errores. El modo historia lleva de a poco de las primeras manos al Viejo Vizcacha.
- **Transmite la cultura:** pulperías por región, mate, payadas, la leyenda de la Salamanca y el Viejo Vizcacha de *Martín Fierro*.
- **Es seguro para jugar en grupo:** el servidor es la única autoridad. Cada cliente recibe solo su propia mano, y toda jugada se valida en el servidor.

# 2. Requisitos

Cada requisito tiene un identificador, una redacción en términos de lo que hace el sistema y la forma de verificarlo. La guía de prueba en vivo (07-pruebas.md) recorre todos los requisitos funcionales en orden.

## 2.1 Requisitos funcionales

### Mesa y partido de truco (núcleo)

| ID | Requisito | Verificación |
|---|---|---|
| RF-01 | El sistema permite colocar una **Mesa de Truco** y hasta cuatro **Sillas Criollas** pegadas a ella. Con clic derecho en una silla, el jugador queda sentado mirando a la mesa. | Colocar mesa y silla; clic derecho en la silla. |
| RF-02 | El jugador sentado abre el **tapete** (clic en la mesa o tecla J) y elige 1 contra 1 o 2 contra 2, partido a 30 o 40 y con o sin flor. Después empieza el partido. | Configurar y apretar «Empezar». |
| RF-03 | Al empezar cada mano, el sistema reparte 3 cartas a cada jugador de un mazo español de 40, da vuelta la **muestra** y marca las **piezas** (2, 4, 5, 10 y 11 del palo de la muestra). | Ver la mano y la muestra; las piezas se resaltan. |
| RF-04 | El tapete muestra solo las acciones **legales** en ese momento. Cada una tiene su atajo de teclado (1 a 3, T, E, R, F, L, C, A, X, Q, N y M). | En cada estado aparecen solo los botones válidos. |
| RF-05 | El sistema permite cantar **envido, real envido y falta envido** (y sus combinaciones), en la primera baza. Calcula los tantos según las reglas uruguayas, con el valor de las piezas. Los tantos se cantan en voz alta empezando por la mano y se anotan. | Cantar envido y comparar los tantos mostrados con el cálculo manual. |
| RF-06 | El sistema permite cantar **flor**, **contraflor**, **contraflor al resto** y **con flor me achico**. «Con flor me achico» suma 3 puntos. | Con una mano de flor, cantar y ver los puntos. |
| RF-07 | El sistema permite cantar **truco, retruco y vale cuatro**, con quiero y no quiero, e **irse al mazo**. En 2 contra 2, si uno se va al mazo se va todo el equipo. | Cantar truco, subirlo, irse al mazo en 2v2. |
| RF-08 | El sistema resuelve cada baza y las **pardas**, decide quién gana la mano y anota los puntos en **malas y buenas** (15/15 o 20/20). El partido termina al llegar al objetivo. | Jugar un partido completo; ver el anotador. |
| RF-09 | La **falta envido** y la **contraflor al resto** valen el partido entero. Si un equipo está en malas y el otro en buenas, valen lo que le falta al que va ganando. | Probar con `/truco` en un partido avanzado o con las pruebas automáticas. |
| RF-10 | Los asientos libres se completan con **paisanos** (NPC) que juegan solos, con cinco niveles de habilidad. El Viejo Vizcacha juega con un sexto nivel. | Empezar con un solo humano. |
| RF-11 | Con clic derecho, el jugador invita a un paisano a jugar. El paisano camina hasta la mesa más cercana (24 bloques como máximo) y se sienta. | Clic derecho (agachado, si tiene un pedido) a un paisano cerca de una mesa. |
| RF-12 | En 2 contra 2, el jugador le hace **señas** a su compañero (tecla G). Si un rival lo ve, se entera. | Partido 2v2 con un compañero; apretar G. |
| RF-13 | Las cartas jugadas, la muestra y los dorsos de las manos se ven **sobre la mesa en el mundo**, para todos los jugadores cercanos. | Mirar la mesa desde afuera con otro jugador. |
| RF-14 | Antes de empezar, los jugadores pueden **apostar** lo que tienen en la mano (esmeraldas, por ejemplo) y retirarlo. En la Guarida Clandestina solo se apuesta oro, esmeraldas, diamantes o netherite. Al terminar, el equipo ganador se lleva el pozo. | Apostar, jugar, cobrar. |

### Progresión y modo historia

| ID | Requisito | Verificación |
|---|---|---|
| RF-15 | Cada partido suma o resta **Prestigio Criollo**. El prestigio define cinco niveles con su rango. `/truco prestigio` muestra el prestigio y las estadísticas. | Ganar y perder; usar el comando. |
| RF-16 | El mundo genera **pulperías** por bioma (y junto a las aldeas), la **Guarida Clandestina**, una sola **Estancia del Caudillo** y el **Paso de las Ánimas**. | Explorar, o usar `/locate trucoya:pulperia_llanura`. |
| RF-17 | La **Libreta del Truquero** tiene cuatro pestañas: capítulos y sellos, barajas, pedidos y desafíos del día, y estadísticas. | Clic derecho con la Libreta. |
| RF-18 | Ganar en la pulpería de cada una de las seis regiones da su **sello**. Con los seis sellos y el nivel 4 llega el **Mapa del Baqueano**, que marca la Estancia. Cada pulpería tiene un **torneo** de 3 rondas. | `/truco completar` acelera el camino. |
| RF-19 | En la Estancia espera el **Viejo Vizcacha**, con una cinemática al conocerlo. Ganarle da la Baraja de Oro y la **cinemática final**, con las voces de los cuatro Fundadores. | `/truco cinematica` y `/truco cinematica final`. |
| RF-20 | El capítulo **La Salamanca**: el Vizcacha viene a buscar al jugador, cabalgan bajo la tormenta hasta el Paso de las Ánimas y juegan un 2 contra 2 contra el Mandinga y el Negro Ledesma. Al amanecer el alma de Ledesma queda libre. | `/truco salamanca visita` (operador). |
| RF-21 | **Mate y picada:** se cultiva yerba, se calienta la pava, se ceba y se convida mate (efecto Confraternidad). Hay picada, bebidas y cuatro discos de música criolla. | Ver la Guía de prueba, sección Fase 5. |
| RF-22 | **Carreras cuadreras** contra paisanos a caballo, entre dos Rayas Cuadreras. | Colocar las rayas y desafiar. |
| RF-23 | Hay **diez barajas coleccionables**. Se ganan con hazañas o se compran al pulpero; se elige con cuál jugar, y la ve toda la mesa. | Pestaña Barajas de la Libreta. |
| RF-24 | **Pedidos de los paisanos:** traer cosas, llevar una encomienda o cumplir un desafío. Hay hasta 3 a la vez, con premio en esmeraldas y prestigio, y un pedido cumplido no se repite. | Paisano con «¡Pedido!»; `/truco pedidos`. |
| RF-25 | **Desafíos del día:** tres por jugador y por día, con premio y racha de días seguidos. | Pestaña Pedidos de la Libreta. |
| RF-26 | Hay un árbol de **logros** (advancements) propio. | Menú de Progresos (L). |

### Juego en red, administración y herramientas

| ID | Requisito | Verificación |
|---|---|---|
| RF-27 | El mod funciona en un **servidor dedicado**. El progreso de cada jugador (prestigio, sellos, barajas, pedidos) queda guardado con su personaje y sobrevive a la muerte y al reinicio. | `scripts\servidor.bat`; entrar con dos clientes. |
| RF-28 | El administrador configura en `serverconfig/trucoya-server.toml` si la Salamanca puede cambiar la hora y el clima, si hay pedidos y la espera de `/truco cinematica`. | Editar el archivo y reiniciar. |
| RF-29 | **Modo tráiler:** el jugador marca lugares y la cámara de cine los recorre con carteles y música. | `/truco trailer marcar`, luego `/truco trailer`. |
| RF-30 | **Comandos de prueba para operadores:** `/truco completar [todo]`, `/truco gaucho <rol> [nivel]`, `/truco salamanca …`, `/truco prestigio set …`. | Probarlos como operador y como no operador. |
| RF-31 | Todos los textos están en **español rioplatense** (es_uy, es_ar, es_es y es_mx). Los nombres de ítems están además en inglés (en_us). | Cambiar el idioma del juego. |

## 2.2 Requisitos no funcionales

### Rendimiento

| ID | Requisito | Criterio medible | Cómo se mide |
|---|---|---|---|
| RNF-R1 | La lógica de las mesas no afecta la fluidez del servidor. | Con 2 mesas en juego y 8 paisanos, el servidor se mantiene en **20 TPS** (≥ 19,5). | `/forge tps` durante un partido. |
| RNF-R2 | Los paisanos responden con un ritmo natural, sin trabar la partida. | Cada decisión de un paisano llega entre **1,0 y 2,25 s** después de su turno (20 a 44 ticks). | Constantes `BOT_MIN_DELAY` y `BOT_EXTRA_DELAY` de `MesaTrucoTileEntity`; se ve al jugar. |
| RNF-R3 | El tráfico de red por mesa es acotado. | Cada vista de mesa ocupa **≤ 16 KB** (`TableViewPacket.MAX_BYTES`). Los globos de diálogo y los avisos ocupan ≤ 1 KB, y los textos se recortan a 200 caracteres. | Límites en el código: lo que se pasa del límite se rechaza al decodificar. |
| RNF-R4 | La inteligencia artificial decide rápido aun en su nivel más alto. | En el nivel 5 (con simulación de Monte Carlo), cada decisión tarda **< 10 ms en promedio y < 250 ms como máximo**, menos que la pausa de 1 s que hace el paisano antes de jugar. Medido en 30 partidos 2v2 entre bots de nivel 5: 3526 decisiones, **3,2 ms** de promedio y **97 a 135 ms** de máximo, según la corrida (el máximo es la primera decisión, antes de que la JVM optimice el código). | `java com.trucoya.core.MedicionIA` (en `src/test`). |

## Seguridad

| ID | Requisito | Criterio medible | Cómo se verifica |
|---|---|---|---|
| RNF-S1 | El servidor es la **única autoridad** del partido. | El cliente solo manda intenciones («jugar la carta X», «cantar truco»). El servidor valida asiento, turno y legalidad (`TrucoMatch.isLegal`) y rechaza el **100 %** de las jugadas ilegales. | `handleAction` y `handleCard` en `MesaTrucoTileEntity`; pruebas automáticas de jugadas ilegales. |
| RNF-S2 | **Nadie ve las cartas de otro.** | Cada cliente recibe solo su `PlayerView`, que trae su propia mano. La vista pública de la mesa solo trae cartas ya jugadas y cuántas le quedan a cada uno. Ningún paquete lleva manos ajenas. | Lectura de `PlayerView` / `vistaTag()`. |
| RNF-S3 | Solo se actúa sobre una mesa estando cerca. | Las acciones de mesa se rechazan a más de **8 bloques**. | `inReach()`; intentar desde lejos. |
| RNF-S4 | Los comandos que alteran el progreso son solo para operadores. | `completar`, `gaucho`, `guardia`, `salamanca`, `sello`, `prestigio set` y `trailer` exigen **permiso nivel 2**. `trailer` también lo puede usar el dueño de un mundo de un jugador. | Probarlos sin ser operador: responden «comando desconocido». |
| RNF-S5 | Los paquetes no se pueden usar para trabar o tumbar el servidor. | Cada paquete limita tamaños y cantidades al decodificar: listas ≤ 8, textos ≤ 1024, índices validados. | Revisión de `decode()` de cada paquete. |
| RNF-S6 | No hay credenciales ni conexiones externas. | El mod no hace ninguna conexión a internet mientras se juega. El repositorio no contiene claves ni tokens. | Búsqueda en el código; análisis estático (08-calidad.md). |
| RNF-S7 | No se pueden explotar las recompensas. | Un pedido cumplido o cancelado no se acepta de nuevo, cada desafío diario paga **una vez**, y las barajas de hazaña no se transfieren. | Pruebas de la guía, sección Pedidos. |

## Usabilidad

| ID | Requisito | Criterio medible |
|---|---|---|
| RNF-U1 | Empezar a jugar es inmediato. | Un jugador nuevo arranca su primer partido en **3 pasos**: poner mesa y silla, sentarse y apretar «Empezar». |
| RNF-U2 | La interfaz no confunde. | El tapete muestra **solo** botones de acciones legales, y el **100 %** tiene atajo de teclado. |
| RNF-U3 | La mesa se sigue viendo mientras se juega. | En la vista compacta, la mano y los botones ocupan **≤ 30 %** de la altura de la pantalla, en la franja inferior; el resto queda libre para ver la mesa. |
| RNF-U4 | El juego habla como el jugador. | El **100 %** de los textos del juego está en español rioplatense. |
| RNF-U5 | El jugador siempre sabe qué hacer después. | La Libreta muestra el próximo paso del modo historia en **toda** etapa. |

## Compatibilidad, mantenibilidad y fiabilidad

| ID | Requisito | Criterio medible |
|---|---|---|
| RNF-C1 | Plataforma | Minecraft Java **1.16.5**, Forge **36.2.39**, GeckoLib **3.0.106** y Java **8**. Probado en Windows 10/11. |
| RNF-M1 | Motor separado | El paquete `com.trucoya.core` no importa ninguna clase de Minecraft. Se compila y se prueba solo con el JDK. |
| RNF-M2 | Pruebas automáticas | **33 pruebas** del motor (`gradlew coreTests`), en la integración continua de GitHub Actions en cada push. |
| RNF-F1 | Tolerancia a fallos | Si una cinemática o un efecto falla, se corta solo esa escena, el juego sigue y el error queda en el log. Una partida o escena cortada (desconexión, cierre) deja el mundo como estaba: la Salamanca vuelve a ser tapera y el jugador vuelve a su lugar. |
| RNF-F2 | Persistencia | Todo el progreso se guarda en formato NBT con el mundo. Al cerrar y abrir el juego, el progreso es el mismo. |

# 3. Stack tecnológico

Truco Ya es un **mod de Minecraft**. Las capas habituales de un sistema (frontend, backend, base de datos, red) existen igual, pero dentro de la arquitectura cliente-servidor de Minecraft: el *frontend* es el cliente del juego y el *backend* es el servidor (integrado si se juega solo, dedicado si se juega en red).

## 3.1 Tecnologías del producto

| Capa | Tecnología (versión) | Para qué se usa en Truco Ya | Por qué se eligió |
|---|---|---|---|
| Lenguaje | **Java 8** (OpenJDK / Temurin 8) | Todo el mod: motor de truco, lógica de servidor y pantallas. | Es el lenguaje y la versión que exige Minecraft 1.16.5. Tiene tipado estático y programación orientada a objetos, que es lo que se trabaja en el curso. |
| Plataforma | **Minecraft Java Edition 1.16.5** | El mundo, el render 3D, los jugadores y el servidor. | Ahí está el público (ver 01-situacion-problema.md). Corre en computadoras modestas y es de las versiones con más mods. |
| Framework (backend y frontend) | **Minecraft Forge 36.2.39** | Registro de bloques, ítems, entidades y sonidos. Bus de eventos, canal de red, configuración del servidor y generación de estructuras. | Es el cargador de mods estándar para 1.16.5. Trae de fábrica red, configuración y estructuras; con Fabric harían falta varias librerías aparte. |
| Mapeos | **Mapeos oficiales de Mojang** (`official 1.16.5`) | Nombres legibles de las clases de Minecraft al programar. | Son los nombres oficiales, publicados por Mojang. |
| Animación 3D | **GeckoLib 3.0.106** | Modelos y animaciones de los paisanos: caminar, sentarse, gestos al cantar, montar. | Es la librería estándar para animaciones complejas en Forge. Los modelos se exportan desde Blockbench. |
| Interfaz gráfica (frontend) | API de pantallas de Minecraft (`Screen`, `TileEntityRenderer`, `RenderSystem`) | El tapete, la Libreta, las señas, las cartas sobre la mesa y las cinemáticas. | Es lo nativo del juego: no agrega dependencias y respeta la escala de interfaz del jugador. |
| Comunicación cliente-servidor | **Forge SimpleChannel** (canal `trucoya:main`, protocolo v7), sobre la conexión TCP de Minecraft | 10 paquetes propios (ver 06-infraestructura.md). | Viaja por la misma conexión que el juego, con versión de protocolo propia. Si cliente y servidor tienen versiones distintas del mod, no se conectan. |
| Persistencia ("base de datos") | **NBT** de Minecraft: `WorldSavedData` (datos del mundo), datos persistentes del jugador y datos de bloques y entidades | Progreso de cada jugador (prestigio, sellos, barajas, pedidos, desafíos), estado de cada mesa, pedidos de cada paisano, lugar del Paso de las Ánimas. | Se guarda junto con el mundo, en el mismo momento que el resto del juego. No requiere servidor de base de datos ni red, y una copia del mundo incluye todo. Se evaluó una base SQL (por ejemplo, SQLite) y se descartó: obliga a sincronizar dos almacenamientos y complica las copias de seguridad, sin necesidad de consultas complejas. |

| Capa | Tecnología (versión) | Para qué se usa en Truco Ya | Por qué se eligió |
|---|---|---|---|
| Configuración del servidor | **ForgeConfigSpec** (TOML) | `serverconfig/trucoya-server.toml`: Salamanca, pedidos y espera de comandos. | Es lo estándar de Forge: valida tipos y rangos, y se guarda con cada mundo. |
| Contenido de datos | JSON de Minecraft: modelos, estados de bloque, logros, recetas, idiomas, sonidos; plantillas de estructuras `.nbt` | Bloques, ítems, recetas, 20+ logros, 5 idiomas y las estructuras del mundo. | Es el formato de recursos y datos de Minecraft. |

## 3.2 Herramientas de construcción y calidad

| Herramienta | Uso | Justificación |
|---|---|---|
| **Gradle 7.6.4** (wrapper incluido) + **ForgeGradle 5.1** | Compilar, ofuscar de vuelta (reobf) y armar el `.jar`. También `coreTests`, `runClient` y `runServer`. | Es el sistema de construcción oficial de Forge. El wrapper baja la versión correcta sin instalar nada. |
| **Pruebas propias del motor** (`TrucoCoreTests`, Java puro) | 33 pruebas automáticas de reglas, IA y serialización. | El motor no depende de Minecraft, así que se prueba en segundos sin abrir el juego ni bajar librerías. |
| **GitHub Actions** | Integración continua: compila y corre las pruebas en cada push. | Gratis para repositorios de GitHub; detecta enseguida si un cambio rompe algo. |
| **PMD 7** (reglas en `config/pmd/reglas.xml`) | Análisis estático: errores probables, seguridad y buenas prácticas. | Herramienta libre, estándar para Java (ver 08-calidad.md). |
| **PlantUML** | Diagramas UML en `docs/uml/*.puml` (texto versionado). | Los diagramas se mantienen como código, junto con el programa. |
| Scripts `.bat` de Windows | `scripts\probar.bat`, `scripts\jugar.bat`, `scripts\actualizar.bat`, `scripts\servidor.bat` y `scripts\armar_jar.bat`. | El grupo trabaja en Windows: con doble clic se baja el JDK, se compila, se juega y se arma el servidor. |

## 3.3 Herramientas de producción de contenido (no van dentro del `.jar`)

Las texturas, estructuras, voces y música del mod se generan con programas propios en Python, en `tools/`. Así todo el contenido es original y se puede volver a generar.

| Herramienta | Uso |
|---|---|
| **Python 3** con **NumPy**, **SciPy** y **Pillow** | Texturas (las 10 barajas de 40 cartas, ítems, bloques) y plantillas de estructuras `.nbt` (pulperías, estancia, Salamanca). |
| **sherpa-onnx** (Kokoro, Piper, Whisper, GTCRN), **ONNX Runtime**, **pyworld**, **librosa** | Voces de los cantos y de los personajes con síntesis neuronal. Control de calidad con reconocimiento de voz (Whisper) y nota de oyente (DNSMOS). Limpieza de las grabaciones reales de los Fundadores (GTCRN). |
| **FFmpeg** | Conversión de audio a OGG Vorbis, el formato de Minecraft. |

| Herramienta | Uso |
|---|---|
| **nbtlib** + `tools/vista_estructuras.py` | Escribir las plantillas de estructuras ( `.nbt` ) y dibujarlas en isométrico, con las texturas de Minecraft, para revisarlas sin abrir el juego. |

Dependencias: `tools/requirements.txt`. Los modelos de voz se descargan solos a `tools/.modelos/`, que no se sube al repositorio.

## 3.4 Correspondencia entre el stack y el código

| Declarado | Dónde está en el código |
|---|---|
| Java 8 | `build.gradle` ( `java.toolchain.languageVersion = 8` ); el JDK 8 lo baja `scripts\probar.bat`. |
| Forge 36.2.39, Minecraft 1.16.5 | `gradle.properties` ( `forge_version` , `minecraft_version` ), `META-INF/mods.toml`. |
| GeckoLib 3.0.106 | `gradle.properties` ( `geckolib_version` ), `entity/GauchoEntity.java` ( `IAnimatable` ), `client/GauchoModel.java` , `client/GauchoRenderer.java`. |
| SimpleChannel | `network/TrucoNetwork.java` y los 10 `*Packet.java`. |
| NBT | `progreso/JugadorData.java` , `progreso/SalamancaSitio.Datos` ( `WorldSavedData` ), `tileentity/MesaTrucoTileEntity.save/load` , `entity/GauchoEntity.addAdditionalSaveData`. |
| ForgeConfigSpec | `TrucoConfig.java` , registrado en `TrucoYa.java`. |
| Pruebas | `src/test/java/com/trucoya/core/TrucoCoreTests.java` , tarea `coreTests` en `build.gradle`. |
| CI | `.github/workflows/build.yml`. |
| PMD | `config/pmd/reglas.xml`. |

# 4. Diagramas UML

Los diagramas están escritos en PlantUML, en `docs/uml/`, y se dibujaron a partir del código real. Las imágenes están en `docs/img/`, en SVG y PNG.

Para volver a dibujarlos:

```
java -jar plantuml.jar -tsvg -o ../img docs/uml/*.puml
```

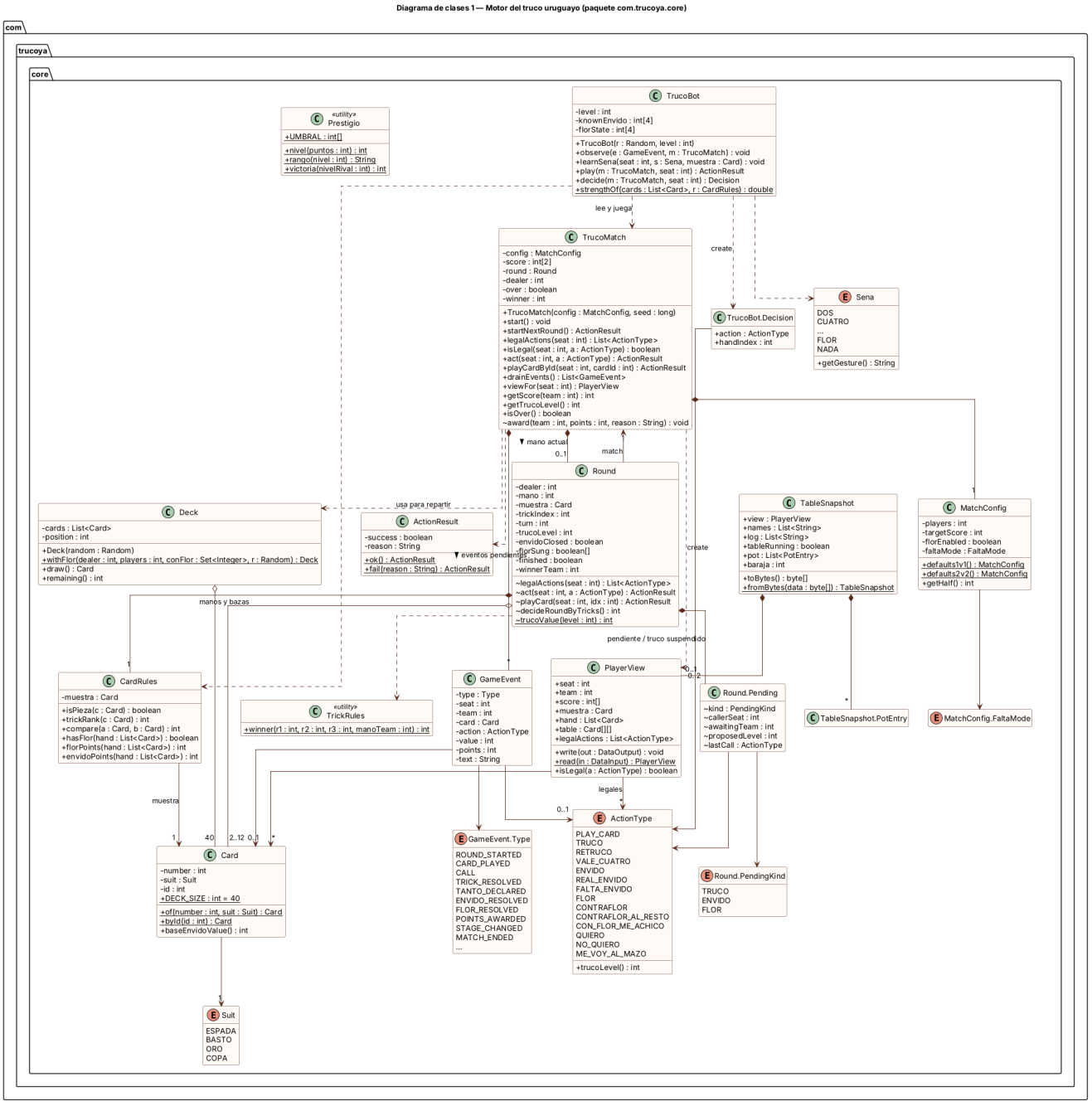
**Notación:**

| Símbolo | Significado |
|---|---|
| `+` | público |
| `-` | privado |
| `#` | protegido |
| `~` | de paquete |
| subrayado | estático |
| ◆ (composición) | la parte vive dentro del todo |
| ◇ (agregación) | la parte existe por su cuenta |
| línea punteada | dependencia / «create» |
| triángulo vacío | herencia o realización |

Las cardinalidades están en cada extremo. Las clases de Minecraft, Forge y GeckoLib aparecen en gris, solo cuando hace falta mostrar la herencia.

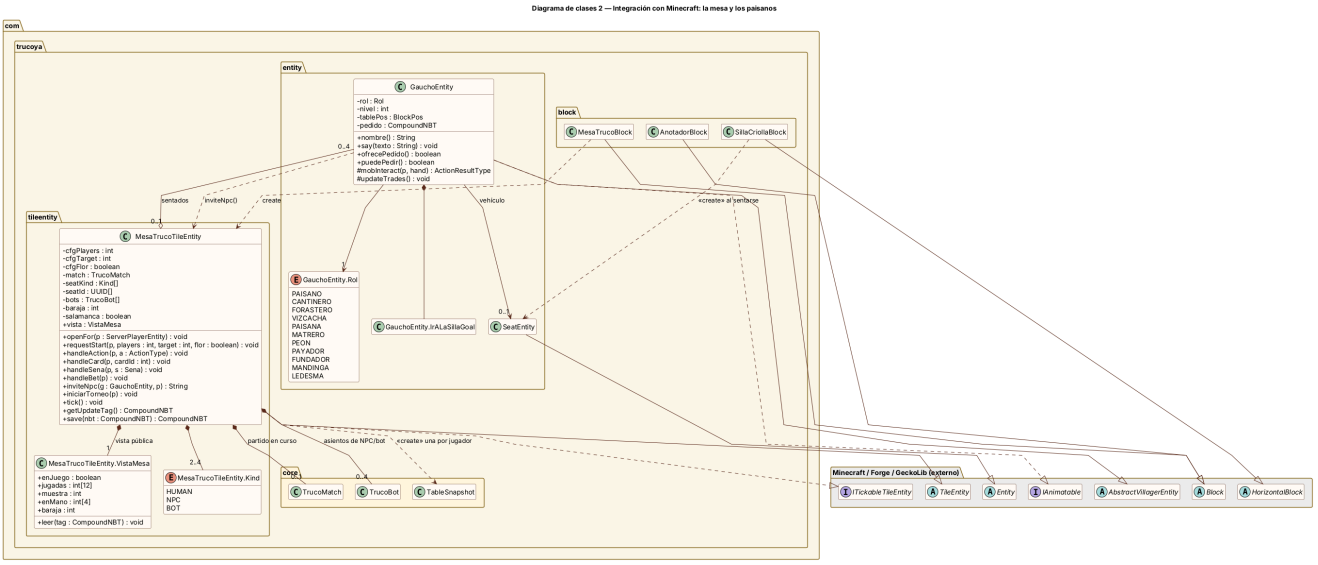
## 4.1 Diagrama de clases 1: el motor del truco (`com.trucoya.core`)

Este es el corazón del sistema y no depende de Minecraft: - `TrucoMatch` lleva el partido completo y crea una `Round` por cada mano. - `Round` aplica todas las reglas de la mano: bazas, envido, flor, truco y pardas. - `CardRules` resuelve la muestra y las piezas. - `TrucoBot` es la inteligencia artificial de los paisanos. - `PlayerView` es lo único que ve cada jugador.



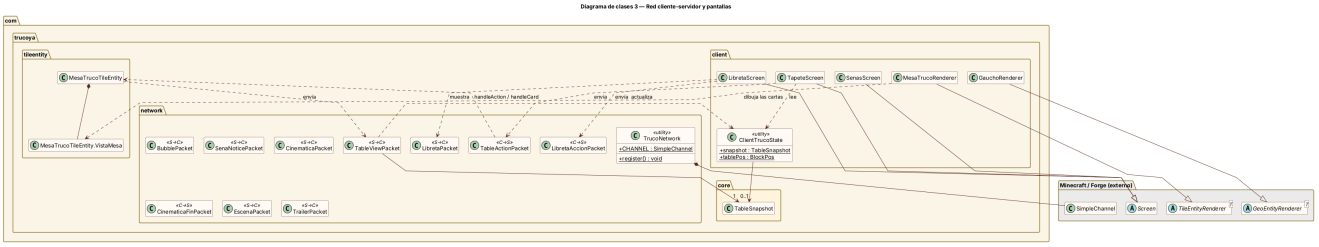

## 4.2 Diagrama de clases 2: la mesa y los paisanos

- `MesaTrucoTileEntity` es el bloque-entidad de la mesa en el servidor. Contiene el `TrucoMatch`, los `TrucoBot` de los asientos que no son humanos y la vista pública (`VistaMesa`) que ven los demás jugadores.
- `GauchoEntity` es el paisano: hereda de `AbstractVillagerEntity` para comerciar e implementa `IAnimatable` de GeckoLib.



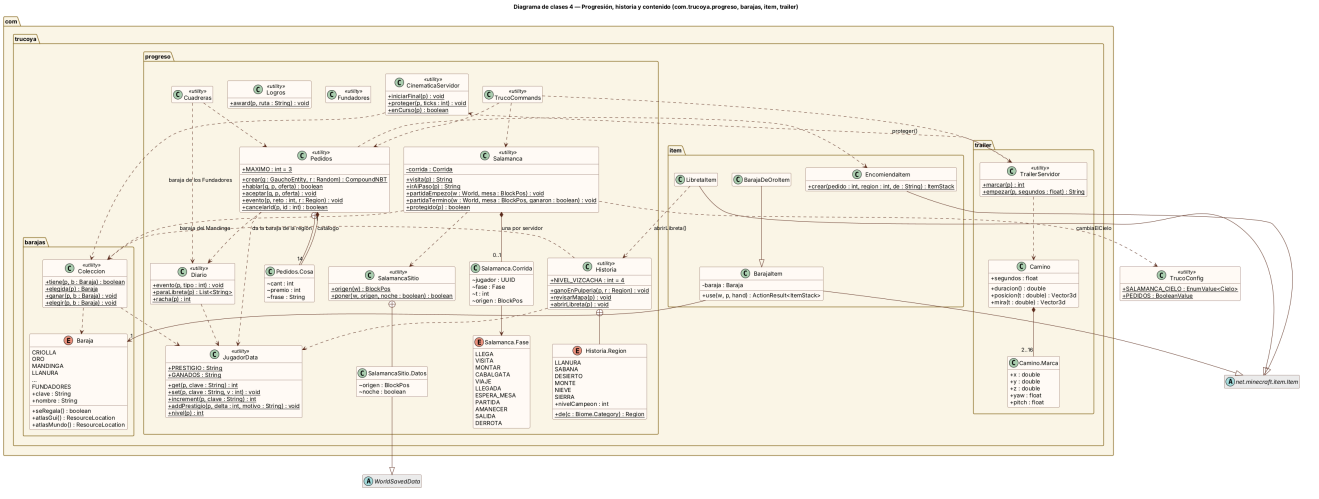

# 4.3 Diagrama de clases 3: red y pantallas

- El cliente solo manda intenciones (`TableActionPacket`).
- El servidor responde con un `TableSnapshot` por jugador (`TableViewPacket`), que el cliente guarda en `ClientTrucoState` y dibuja en `TapeteScreen`.
- Las cartas sobre la mesa en el mundo las dibuja `MesaTrucoRenderer` a partir de `VistaMesa`.



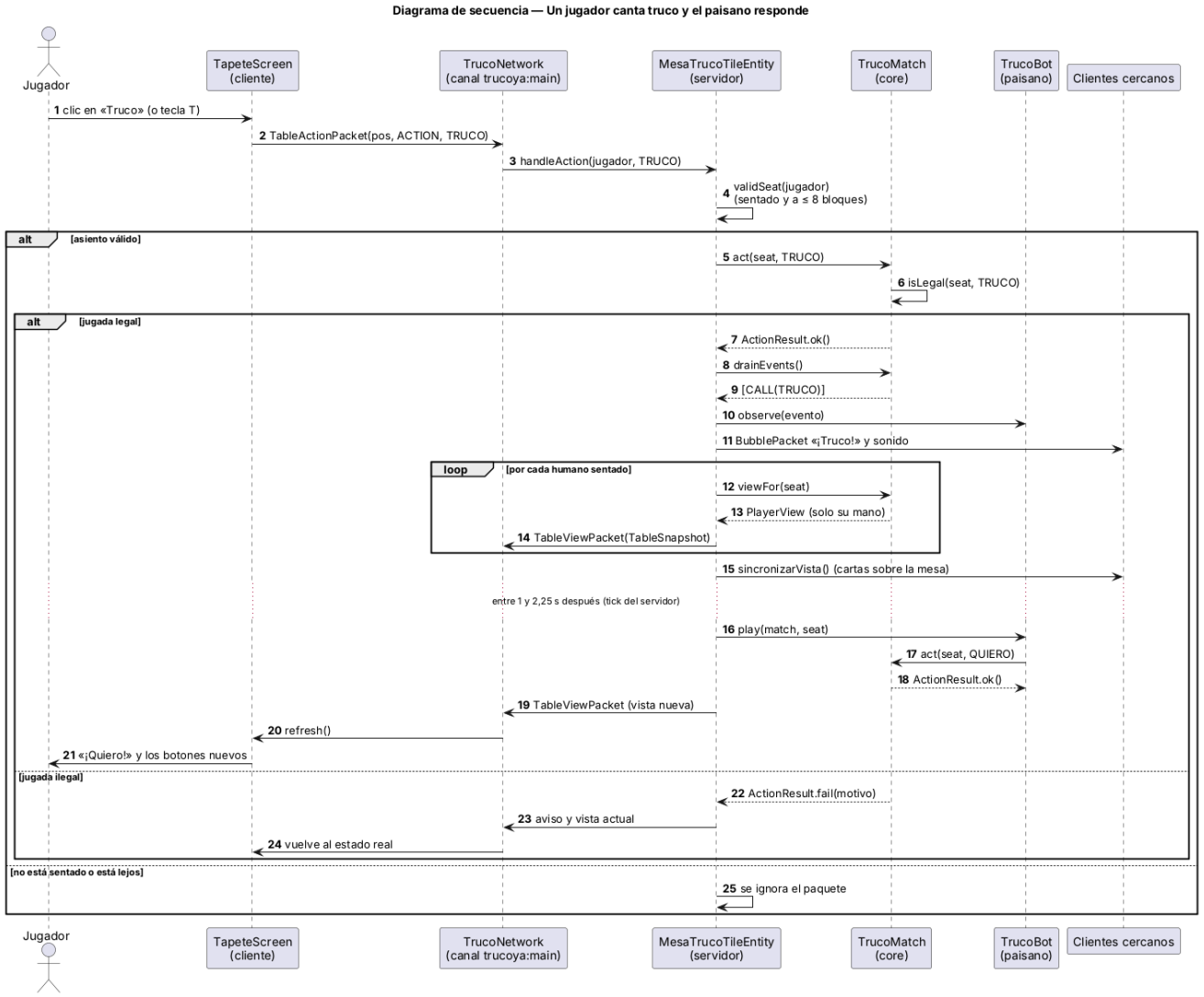

# 4.4 Diagrama de clases 4: progresión, historia y contenido

- El progreso de cada jugador vive en sus datos persistentes (`JugadorData`, NBT).
- `Historia` maneja regiones, sellos y la Libreta.
- `Pedidos`, `Diario` y `Coleccion` son los pedidos, los desafíos y las barajas.
- `Salamanca` es una máquina de estados (`Fase`), con una sola corrida por servidor.
- `TrailerServidor` y `Camino` forman el modo tráiler.



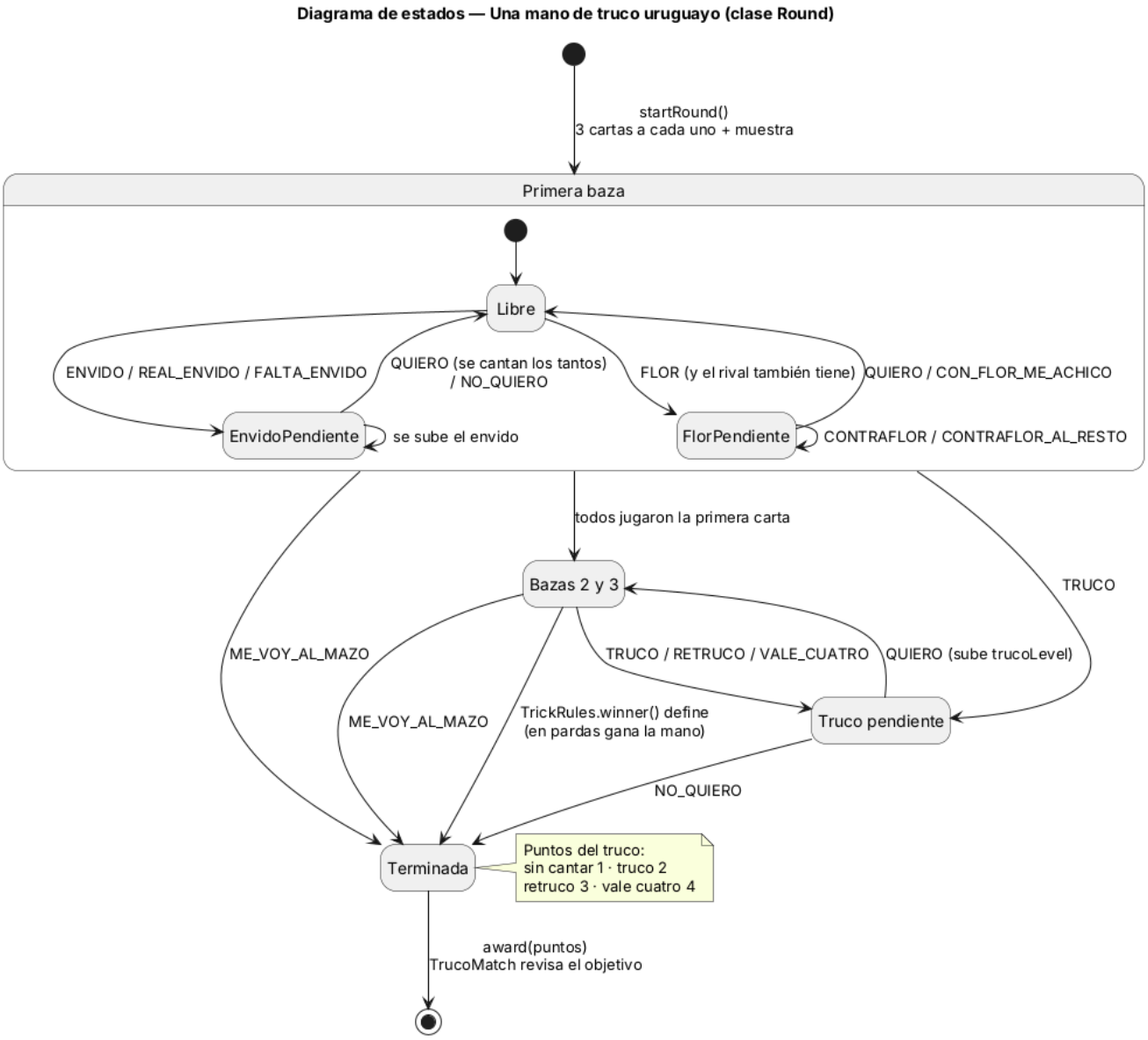
## 4.5 Diagrama de secuencia: cantar truco

Muestra el viaje completo de una jugada: el clic en el cliente, la validación en el servidor, la actualización privada a cada jugador y la respuesta del paisano.



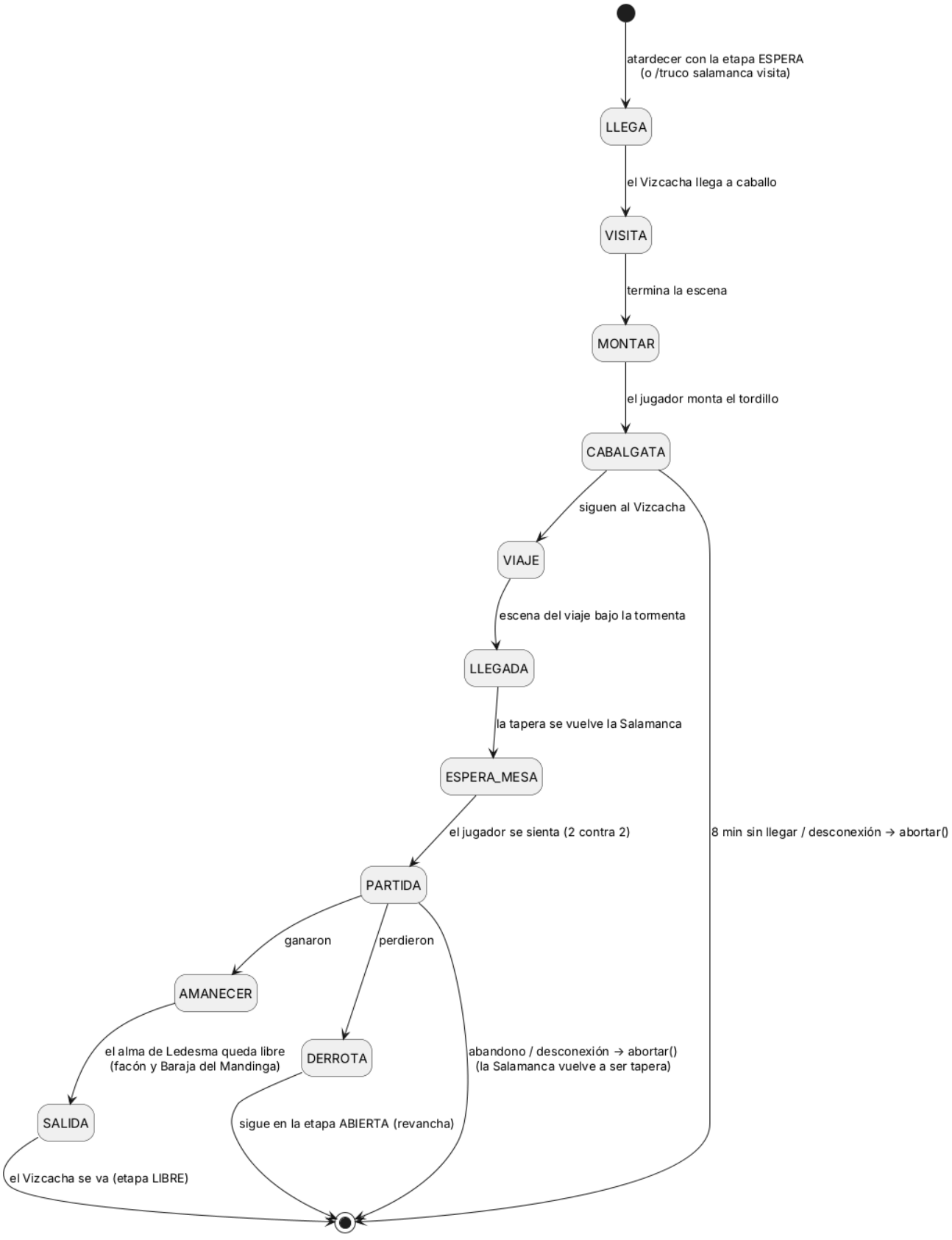

## 4.6 Diagramas de estados

### Una mano (`Round`):




## El capítulo La Salamanca (`Salamanca.Fase`):

**Diagrama de estados — Capítulo «La Salamanca» (enum Salamanca.Fase)**

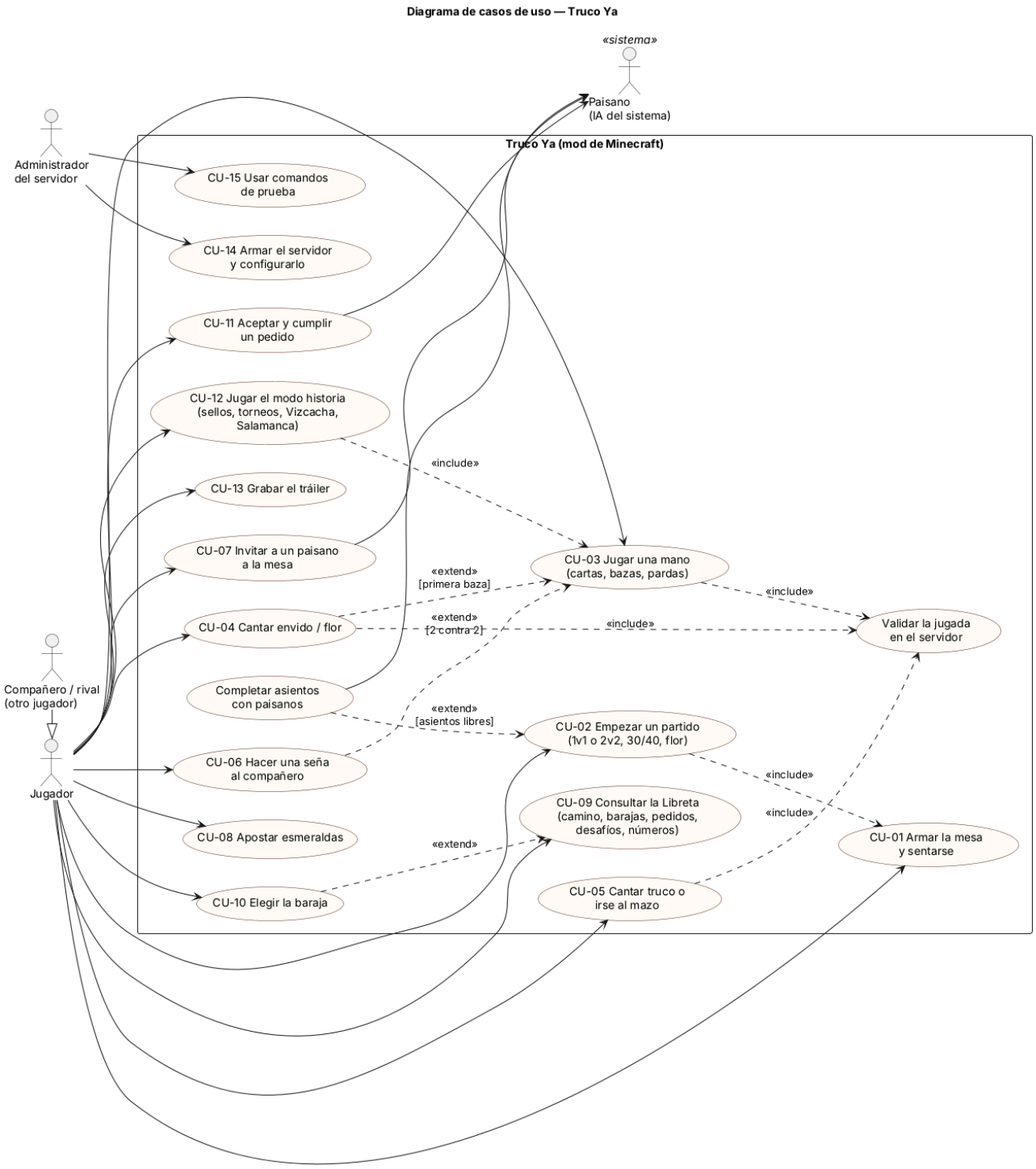


# 5. Casos de uso

## 5.1 Actores

| Actor | Tipo | Descripción |
|---|---|---|
| **Jugador** | Principal | Persona que juega Minecraft con el mod instalado. |
| **Compañero / rival** | Principal (es un Jugador) | Otro jugador en la misma mesa, en un servidor. |
| **Administrador del servidor** | Principal | Instala y configura el servidor dedicado; es operador (permiso nivel 2). |
| **Paisano (IA)** | Secundario, del sistema | NPC que juega, acepta invitaciones y ofrece pedidos. |

## 5.2 Diagrama




Fuente: `uml/casos_de_uso.puml`.

# 5.3 Descripciones

## CU-01 Armar la mesa y sentarse

- **Actor:** Jugador. **Requisitos:** RF-01.

- **Precondición:** el jugador tiene una Mesa de Truco y al menos una Silla Criolla (se compran al pulpero o se fabrican).
- **Flujo principal:** 1. El jugador coloca la mesa. 2. Coloca una silla pegada a uno de sus lados. 3. Hace clic derecho en la silla. 4. El sistema lo sienta mirando hacia la mesa.
- **Postcondición:** el jugador ocupa un asiento de esa mesa.

### CU-02 Empezar un partido

- **Actor:** Jugador. **Requisitos:** RF-02, RF-10.
- **Precondición:** el jugador está sentado (incluye CU-01) y la mesa no tiene un partido en curso.
- **Flujo principal:** 1. El jugador abre el tapete (clic en la mesa o tecla J). 2. Elige modo (1v1 o 2v2), puntos (30 o 40) y si se juega con flor. 3. Aprieta «Empezar». 4. El sistema completa los asientos libres con paisanos (*extend*: «Completar asientos con paisanos»). 5. El sistema reparte y muestra la mano.
- **Alternativos:**
- 3a. Si hay 3 o 4 humanos sentados y se eligió 1 contra 1, el sistema pide elegir 2 contra 2.
- 3b. En la Salamanca solo puede empezar el jugador de esa noche, y hacen falta los cuatro sentados.
- 3c. En la Guarida Clandestina hace falta nivel 3 de prestigio o una Ficha Clandestina.
- **Postcondición:** hay un `TrucoMatch` en curso, con la primera mano repartida y la muestra a la vista.

### CU-03 Jugar una mano

- **Actor:** Jugador; Paisano (rival o compañero). **Requisitos:** RF-03, RF-04, RF-08, RF-13.
- **Precondición:** hay un partido en curso y es el turno del jugador.
- **Flujo principal:** 1. El jugador elige una carta (clic o tecla 1 a 3). 2. El cliente manda la intención al servidor. 3. El sistema valida asiento, distancia y turno (*include*: «Validar la jugada en el servidor»). 4. El sistema pone la carta en la baza. 5. El sistema actualiza la vista privada de cada jugador y la mesa en el mundo. 6. Al completarse la baza, el sistema la resuelve. 7. Al decidirse la mano, el sistema anota los puntos.
- **Alternativos:**
- 3a. Jugada ilegal o fuera de turno: el sistema la rechaza con un mensaje y no cambia nada.
- 6a. Parda: la baza no es de nadie y se sigue según las reglas de pardas.
- **Postcondición:** la carta queda jugada y el estado es el mismo en el servidor y en todos los clientes.

### CU-04 Cantar envido o flor

- **Actor:** Jugador. **Requisitos:** RF-05, RF-06, RF-09.
- **Precondición:** primera baza (*extend* de CU-03), envido o flor todavía abiertos y el canto es legal en ese momento.
- **Flujo principal:** 1. El jugador aprieta «Envido», «Real envido», «Falta envido» o «Flor». 2. El sistema valida el canto (*include*). 3. El rival responde «Quiero», «No quiero» o sube el canto. 4. Si se quiere, los jugadores cantan sus tantos empezando por la mano. 5. El sistema calcula los tantos (con piezas), decide el ganador y anota.
- **Alternativos:**
- 1a. Con flor en la mano, el envido no está disponible: hay que cantar flor.
- 3a. «Con flor me achico»: suma 3 puntos al que cantó la flor.
- 3b. Contraflor al resto con un equipo en malas y el otro en buenas: vale lo que le falta al que va ganando.

- **Postcondición:** el envido o la flor quedan cerrados para esa mano, y los puntos anotados en malas o buenas.

### CU-05 Cantar truco o irse al mazo

- **Actor:** Jugador. **Requisitos:** RF-07.
- **Precondición:** partido en curso y el jugador tiene derecho a cantar o subir.
- **Flujo principal:** 1. El jugador canta «Truco». 2. El sistema valida (*include*). 3. El rival quiere, no quiere o sube a «Retruco». 4. El jugador puede subir a «Vale cuatro». 5. Al terminar la mano, el sistema anota según el nivel querido.
- **Alternativos:**
- 3a. «No quiero»: el que cantó se lleva los puntos del nivel anterior y la mano termina.
- 1a. «Me voy al mazo»: el equipo pierde la mano. En 2v2, irse uno vale por todo el equipo.
- **Postcondición:** el nivel del truco de la mano queda actualizado, o la mano termina.

### CU-06 Hacer una seña al compañero

- **Actor:** Jugador; Compañero. **Requisitos:** RF-12.
- **Precondición:** partido 2v2 en curso.
- **Flujo principal:** 1. El jugador aprieta G. 2. Elige la seña (por ejemplo, «guiño de ojo» por el 2 de la muestra). 3. El sistema se la muestra a su compañero.
- **Alternativo:** 3a. Con cierta probabilidad un rival la ve, y el sistema se lo avisa también a él.
- **Postcondición:** el compañero sabe qué carta importante tiene el jugador.

### CU-07 Invitar a un paisano a la mesa

- **Actor:** Jugador; Paisano. **Requisitos:** RF-11.
- **Precondición:** hay una Mesa de Truco con sillas libres a 24 bloques o menos del paisano.
- **Flujo principal:** 1. El jugador hace clic derecho al paisano (agachado, si el paisano tiene un pedido). 2. El paisano contesta. 3. El paisano camina hasta la mesa y se sienta.
- **Alternativo:** 1a. No hay mesa cerca: el paisano le dice que ponga una.
- **Postcondición:** el paisano ocupa un asiento de la mesa.

### CU-08 Apostar esmeraldas

- **Actor:** Jugador. **Requisitos:** RF-14.
- **Precondición:** el jugador está sentado, la mesa no está jugando y tiene en la mano lo que apuesta (por ejemplo, esmeraldas; en la Guarida, solo oro, esmeraldas, diamantes o netherite).
- **Flujo principal:** 1. El jugador aprieta «Apostar». 2. El sistema pone lo apostado en el pozo y muestra quién apostó. 3. Al terminar el partido, el sistema le da el pozo al equipo ganador.
- **Alternativo:** 2a. «Retirar» antes de empezar devuelve lo apostado. Solo funciona estando en la mesa.
- **Postcondición:** lo apostado queda en el pozo o vuelve al jugador.

### CU-09 Consultar la Libreta del Truquero

- **Actor:** Jugador. **Requisitos:** RF-17, RF-25.
- **Precondición:** el jugador tiene la Libreta (se recibe al entrar al mundo; `/truco historia` la repone).

- **Flujo principal:** 1. El jugador hace clic derecho con la Libreta. 2. El servidor manda los datos del jugador. 3. El sistema muestra cuatro pestañas: Camino (capítulos, sellos y rumbo), Barajas, Pedidos (y desafíos del día) y Números.
- **Postcondición:** el jugador ve su progreso; no cambia nada.

### CU-10 Elegir la baraja

- **Actor:** Jugador. **Requisitos:** RF-23.
- **Precondición:** el jugador tiene la baraja en su colección (*extend* de CU-09).
- **Flujo principal:** 1. En la pestaña Barajas, el jugador toca un mazo. 2. Aprieta «Jugar con esta». 3. El servidor valida que la tiene y la elige.
- **Alternativos:**
- 1a. Clic derecho con el mazo en la mano, o `/truco baraja <clave>`.
- 3a. Si no la tiene, el sistema dice cómo se gana.
- **Postcondición:** los próximos partidos que arme el jugador usan esa baraja.

### CU-11 Aceptar y cumplir un pedido

- **Actor:** Jugador; Paisano. **Requisitos:** RF-24.
- **Precondición:** un paisano muestra «¡Pedido!» y el jugador tiene menos de 3 pedidos.
- **Flujo principal:** 1. El jugador hace clic derecho al paisano. 2. El paisano cuenta qué necesita y el premio. 3. El jugador vuelve a hacer clic (o aprieta [Aceptar] en el chat). 4. El sistema anota el pedido. 5. El jugador lo cumple: le lleva las cosas, entrega la encomienda en la región indicada o hace el desafío. 6. El sistema le paga esmeraldas y prestigio.
- **Alternativos:**
- 5a. Le faltan cosas: el paisano dice cuántas faltan.
- 5b. El jugador cancela el pedido desde la Libreta o con `/truco pedidos cancelar <n>`.
- **Postcondición:** el pedido queda cumplido o cancelado, y no se puede volver a aceptar.

### CU-12 Jugar el modo historia

- **Actor:** Jugador. **Requisitos:** RF-15, RF-16, RF-18, RF-19, RF-20, RF-22.
- **Precondición:** el jugador empezó con su Libreta.
- **Flujo principal:** 1. El jugador gana su primer partido. 2. Busca las pulperías de las seis regiones y gana en cada una (*include* CU-03) para conseguir los sellos. 3. Sube a nivel 4 de prestigio. 4. Recibe el Mapa del Baqueano. 5. Llega a la Estancia y ve la cinemática. 6. Le gana al Viejo Vizcacha y recibe la Baraja de Oro. 7. Ve la cinemática final. 8. Otro día, el Vizcacha lo busca para La Salamanca.
- **Alternativos:**
- 6a. Pierde: puede volver a desafiarlo.
- 8a. Pierde en la Salamanca: queda la revancha.
- **Postcondición:** el avance queda guardado en los datos del jugador.

### CU-13 Grabar el tráiler

- **Actor:** Jugador (operador o dueño del mundo). **Requisitos:** RF-29.
- **Precondición:** el jugador no está en otra escena.
- **Flujo principal:** 1. En cada lugar, el jugador mira hacia donde quiere que mire la cámara y escribe `/truco trailer marcar`. 2. Escribe `/truco trailer 6`. 3. El sistema recorre las marcas con la cámara de cine, carteles y música. 4. Al final, el sistema devuelve al jugador a su lugar.
- **Alternativo:** 2a. Sin marcas, la cámara da una vuelta alrededor del jugador.

- **Postcondición:** el jugador está donde empezó; puede grabar la pantalla.

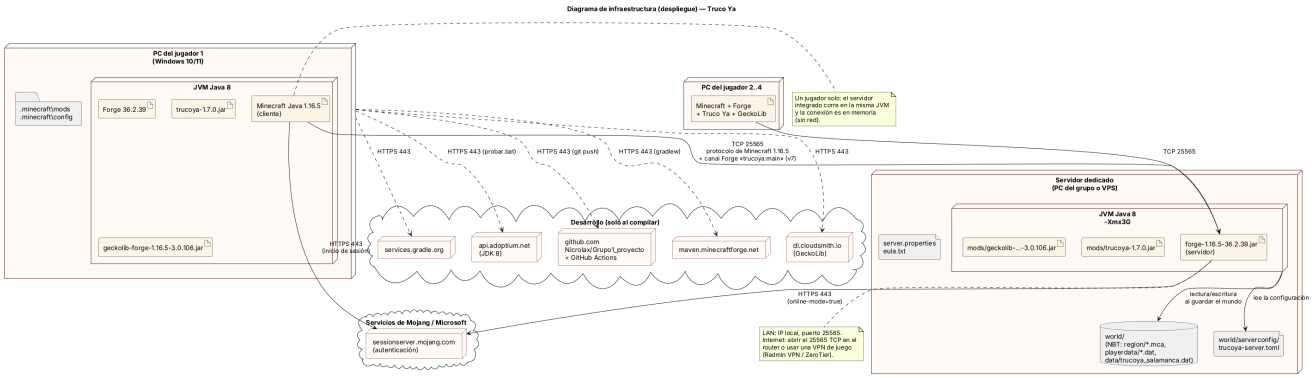
### CU-14 Armar el servidor y configurarlo

- **Actor:** Administrador. **Requisitos:** RF-27, RF-28.
- **Precondición:** PC con Windows y conexión a internet.
- **Flujo principal:** 1. El administrador corre `scripts\servidor.bat`. 2. El script compila el mod, instala Forge para servidor, consigue GeckoLib y pregunta por la EULA de Minecraft. 3. El administrador arranca con `servidor\iniciar.bat`. 4. Si quiere, edita `trucoya-server.toml`. 5. Comparte la IP.
- **Alternativo:** 2a. Si la compilación falla, el script se detiene y no copia nada viejo.
- **Postcondición:** el servidor acepta jugadores con la misma versión del mod.

### CU-15 Usar comandos de prueba

- **Actor:** Administrador (operador). **Requisitos:** RF-30.
- **Precondición:** permiso nivel 2.
- **Flujo principal:** 1. El operador escribe, por ejemplo, `/truco completar` o `/truco gaucho vizcacha`. 2. El sistema aplica el cambio y lo confirma.
- **Alternativo:** 1a. Sin permiso, el comando no aparece y Minecraft lo rechaza.
- **Postcondición:** el progreso o el mundo quedan como pidió el operador.

# 6. Diagrama de infraestructura




Fuente: [uml/infraestructura.puml](uml/infraestructura.puml).

## 6.1 Componentes de despliegue

| Componente | Dónde corre | Qué contiene |
|---|---|---|
| **Cliente** | PC de cada jugador (Windows 10/11), JVM Java 8 | Minecraft Java 1.16.5, Forge 36.2.39, `trucoya-1.7.4.jar` y `geckolib-forge-1.16.5-3.0.106.jar` en `.minecraft\mods`. Dibuja el mundo, las pantallas y el audio. |
| **Servidor dedicado** | PC del grupo o un VPS, JVM Java 8 ( `-Xmx3G` ) | `forge-1.16.5-36.2.39.jar` (servidor), con el mod y GeckoLib en `mods/` . Es la **autoridad** de todo: partidos, IA, progreso. |
| **Servidor integrado** | La misma JVM del cliente, al jugar solo | Igual que el dedicado, pero comunicado en memoria (sin red). |
| **Almacenamiento** ("base de datos") | Disco del servidor, carpeta `world/` | Formato NBT: `region/*.mca` (bloques y mesas), `playerdata/*.dat` (el progreso de cada jugador va dentro de `PlayerPersisted` ) y `data/trucoya_salamanca.dat` ( `WorldSavedData` ). |
| **Configuración** | `world/serverconfig/trucoya-server.toml` , `server.properties` , `eula.txt` | Opciones del mod y del servidor. |
| **Autenticación** | `sessionserver.mojang.com` (Mojang/Microsoft) | Verifica la cuenta de cada jugador al entrar ( `online-mode=true` ). |
| **Repositorio e integración continua** | `github.com/Nicrolax/Grupo1_proyecto` y GitHub Actions | Código, documentación; compila y prueba en cada push. |
| **Repositorios de construcción** | `maven.minecraftforge.net` , `dl.cloudsmith.io` , `services.gradle.org` , `api.adoptium.net` | Solo para compilar: Forge, GeckoLib, Gradle y el JDK 8. No intervienen al jugar. |

## 6.2 Comunicaciones, protocolos y puertos

| Origen → destino | Protocolo | Puerto | Para qué |
|---|---|---|---|
| Cliente → servidor | TCP, protocolo de Minecraft 1.16.5 con el *handshake* de Forge | **25565** (por defecto, en `server.properties`) | Todo el juego. Por la misma conexión viaja el canal propio **`trucoya:main`** (Forge SimpleChannel, protocolo **v7**). Si el mod no coincide en versión, el cliente no entra. |
| Cliente y servidor → Mojang | HTTPS | 443 | Inicio de sesión y verificación de la cuenta. |
| PC de desarrollo → repositorios Maven, Gradle y Adoptium | HTTPS | 443 | Bajar dependencias al compilar (`gradlew`, `scripts\probar.bat`). |
| PC de desarrollo → GitHub | HTTPS | 443 | `git push`; GitHub Actions compila en la nube. |
| Un jugador solo | En memoria | — | El servidor integrado comparte la JVM con el cliente. |

## 6.3 Mensajes del canal `trucoya:main`

| Paquete | Dirección | Contenido | Límites |
|---|---|---|---|
| `TableActionPacket` | Cliente → servidor | Intención en una mesa: abrir, empezar, jugar carta, cantar, seña, apostar, torneo. | La posición de la mesa y los valores se validan en el servidor. |
| `TableViewPacket` | Servidor → cliente | `TableSnapshot`: la vista privada del jugador (`PlayerView`) y la mesa. | ≤ 16 KB |
| `BubblePacket` | Servidor → cliente | Globo de diálogo y canto (con voz). | ≤ 1 KB, 200 caracteres |
| `SenaNoticePacket` | Servidor → cliente | Aviso de seña. | ≤ 1 KB, 160 caracteres |
| `LibretaPacket` | Servidor → cliente | Progreso para la Libreta. | Listas ≤ 8, textos ≤ 1024 |
| `LibretaAccionPacket` | Cliente → servidor | Elegir baraja o cancelar un pedido. | Validado en el servidor |
| `CinematicaPacket`, `EscenaPacket`, `TrailerPacket` | Servidor → cliente | Empezar o cortar una cinemática, una escena o el tráiler. | ≤ 16 marcas |
| `CinematicaFinPacket` | Cliente → servidor | El cliente terminó de ver la escena. | — |

Además, la mesa sincroniza su vista pública (cartas jugadas, muestra, dorsos) con el paquete estándar de bloque-entidad de Minecraft (`SUpdateTileEntityPacket`), solo cuando cambia.

## 6.4 Formas de desplegar

1. **Un jugador:** `scripts\jugar.bat` (desarrollo), o el `.jar` en `.minecraft\mods` con Forge instalado.
2. **LAN:** "Abrir en LAN" desde el juego, o servidor dedicado, con la IP local y el puerto 25565.
3. **Internet:** servidor dedicado (`scripts\servidor.bat`, ver SERVIDOR.md), con el puerto 25565/TCP abierto en el router o con una VPN de juego (Radmin VPN, ZeroTier).

# 7. Pruebas

## 7.1 Pruebas automáticas del motor

```
gradlew coreTests                (con Gradle)
```

Sin Gradle, solo con el JDK:

```
javac -encoding UTF-8 -d out src/main/java/com/trucoya/core/*.java
src/test/java/com/trucoya/core/*.java
java -cp out com.trucoya.core.TrucoCoreTests
java -cp out com.trucoya.core.MedicionIA      (tiempo de decisión de la IA, RNF-R4)
```

Las 33 pruebas cubren: - mazo de 40; - piezas, jerarquía de cartas y muestra; - cálculo de envido y flor; - truco, retruco y vale cuatro, querido y no querido; - cadenas de envido y falta envido; - contraflor, al resto y "con flor me achico"; - pardas y me voy al mazo; - malas, buenas y fin de partido; - turnos en 2 contra 2; - **rechazo de jugadas ilegales** y **vista privada** (RNF-S1, RNF-S2); - serialización de red; - partidos completos entre bots sin jugadas ilegales; - los niveles 1 a 6 de la IA; - torneo, prestigio y el mazo armado de la Salamanca.

Se corren solas en GitHub Actions en cada push (`.github/workflows/build.yml`). Resultado actual: **33 de 33 correctas**.

## 7.2 Guía de prueba en vivo (para el docente)

**Preparación:** 1. Instalar Forge 1.16.5-36.2.39 y copiar a `.minecraft\mods` el `.jar` de Truco Ya (de Modrinth, o el que arma `scripts\armar_jar.bat` en `entrega/`) y el de GeckoLib. En la PC de desarrollo, `scripts\jugar.bat` hace lo mismo. 2. Crear un mundo nuevo en **Creativo, con trucos activados**. Así se pueden usar los comandos de operador para ir rápido.

| # | Prueba | Pasos | Resultado esperado | RF |
|---|---|---|---|---|
| 1 | Armar la mesa | Buscar en el inventario creativo la pestaña «Truco Ya». Poner la Mesa de Truco y dos Sillas Criollas pegadas, y clic derecho en una silla. | El jugador queda sentado mirando a la mesa. | RF-01 |
| 2 | Empezar | Tecla J, 1 contra 1, a 30, con flor, «Empezar». | Se reparten tres cartas y aparece la muestra. Las piezas se destacan y se sienta un paisano. | RF-02, RF-03, RF-10 |
| 3 | Solo lo legal | Mirar los botones en cada momento. | Solo aparecen los botones válidos. «Envido» desaparece después de la primera baza y cada botón muestra su atajo. | RF-04 |
| 4 | Envido | Cantar «Envido» (E). | El paisano responde. Si quiere, se cantan los tantos desde la mano (el que no gana dice «son buenas»); el ganador queda en dorado y los puntos en el anotador. | RF-05 |
| 5 | Truco | Cantar «Truco» (T) y jugar la mano. | El paisano quiere, no quiere o sube. Los puntos se anotan según el nivel. | RF-07, RF-08 |

| # | Prueba | Pasos | Resultado esperado | RF |
|---|---|---|---|---|
| 6 | Mesa en el mundo | Tecla V (vista compacta / clásica) o salir del tapete y mirar la mesa. | Las cartas jugadas están apoyadas frente a cada silla y la muestra sobre el mazo. | RF-13 |
| 7 | Partido completo | Jugar hasta 30, o 20 en un torneo. | Malas y buenas se marcan con fósforos y el partido termina; sube el prestigio (`/truco prestigio`). | RF-08, RF-15 |
| 8 | Flor | Jugar hasta que toque una flor (o ver la prueba automática «Contraflor, al resto y achique»). | Aparece «Flor» y el envido queda bloqueado. | RF-06, RF-09 |
| 9 | 2 contra 2 y señas | Cuatro sillas, 2 contra 2, tecla G. | Los cuatro asientos se completan; la seña le llega al compañero. | RF-12 |
| 10 | Invitar | Dar con `/truco gaucho paisano` y hacer clic derecho al paisano (agachado) cerca de la mesa. | El paisano camina hasta la mesa y se sienta. | RF-11 |
| 11 | Apuesta | Con esmeraldas en la mano, sentado, «Apostar». Después jugar. | El pozo aparece en el tapete y lo cobra el ganador. | RF-14 |
| 12 | Libreta | Clic derecho con la Libreta; recorrer las cuatro pestañas. | Camino, Barajas, Pedidos (con los desafíos del día) y Números. | RF-17, RF-25 |
| 13 | Historia rápida | `/truco completar`, luego `/locate trucoya:estancia` y viajar hasta ahí. | Llegan los seis sellos y el Mapa del Baqueano. En la Estancia, la cinemática del Vizcacha. | RF-16, RF-18, RF-19 |
| 14 | Final | `/truco cinematica final`. | La película final con las **voces reales** de Nicolás, Agustín, Matías y Thiago. | RF-19 |
| 15 | Salamanca | `/truco salamanca visita`. | Llega el Vizcacha a caballo, la cabalgata con tormenta, el Paso de las Ánimas y la partida 2v2. | RF-20 |
| 16 | Mate | Pava al fuego, mate con yerba en la otra mano, cebar; clic derecho a un paisano con el mate cebado. | El paisano agradece y aparece el efecto Confraternidad. | RF-21 |
| 17 | Barajas | Pestaña Barajas, tocar otra baraja (con `/truco completar todo` ya están casi todas) y «Jugar con esta». Empezar un partido. | Las cartas del tapete y de la mesa cambian de diseño. | RF-23 |
| 18 | Pedidos | Buscar un paisano con «¡Pedido!» (o esperar un día), clic derecho dos veces. Cumplirlo. | El pedido aparece en la Libreta; al cumplirlo da esmeraldas y no se puede aceptar de nuevo. | RF-24 |
| 19 | Logros | Tecla L. | Aparece el árbol «Truco Ya» con los logros ganados. | RF-26 |
| 20 | Tráiler | Dos veces `/truco trailer marcar` en lugares distintos, después `/truco trailer 5`. | La cámara recorre las marcas con carteles y música, y el jugador vuelve a su lugar. | RF-29 |
| 21 | Servidor | `scripts\servidor.bat`, `servidor\iniciar.bat`, entrar con dos clientes y sentarse a la misma mesa. | Cada uno ve solo sus cartas y la jugada de uno le llega al otro. | RF-27 |
| 22 | Permisos | Desde un cliente sin operador: `/truco completar`. | El comando no existe para ese jugador. | RF-30, RNF-S4 |
| 23 | Configuración | Poner `cambiaElCielo = "NUNCA"` en `trucoya-server.toml`, reiniciar y hacer `/truco salamanca visita`. | La historia sigue, pero no cambian la hora ni el clima. | RF-28 |
| 24 | Idioma | Opciones → Idioma → English. | Los ítems tienen nombre en inglés. | RF-31 |

## 7.3 Pruebas de seguridad y robustez

| Prueba | Cómo | Esperado |
|---|---|---|
| Jugada ilegal | Prueba automática «Rechazo de jugadas ilegales». | `ActionResult.fail`; el estado no cambia. |
| Mano ajena | Prueba automática «Vista privada del jugador». | La `PlayerView` de un asiento no trae cartas de otro. |
| Acción a distancia | Alejarse más de 8 bloques e intentar retirar una apuesta. | No pasa nada. |
| Desconexión en la Salamanca | Cerrar el juego en medio de la partida y volver a entrar. | La Salamanca vuelve a ser tapera; el capítulo se puede retomar. |
| Pedido repetido | Cumplir un pedido y volver a hacerle clic al paisano, o apretar de nuevo [Aceptar] en el chat. | El paisano no lo vuelve a ofrecer; [Aceptar] responde «Ese pedido ya lo tenés». |

# 8. Calidad del código y análisis estático

## 8.1 Análisis con PMD

Antes de la entrega, el código Java se analizó con **PMD 7.7.0**. El conjunto de reglas está en `config/pmd/reglas.xml` e incluye: - todas las de *errorprone* (errores probables); - todas las de *security*; - reglas de *bestpractices*: variables, campos, métodos y parámetros sin usar, `printStackTrace`, `System.out` y conservar la causa de las excepciones; - reglas de *design* y *codestyle*: excepciones genéricas, `if` booleanos simplificables e imports sin usar.

Para correrlo:

```
pmd check -d src/main/java -R config/pmd/reglas.xml -f text
```

### Resultado

| | Antes | Después |
|---|---|---|
| Hallazgos de prioridad 1 y 2 de seguridad | 0 | 0 |
| Variables, campos e imports sin usar | 6 | 0 |
| `toLowerCase()` sin `Locale` | 5 | 0 |
| Excepciones capturadas sin registrar (`catch` vacío) | 6 | 0 (ahora se registran en el log) |
| Hallazgos menores de estilo que quedan | — | 17 |

Los 17 hallazgos que quedan son de estilo y no cambian el funcionamiento:

- **Asignaciones dentro de una condición** (4). Por ejemplo, `while ((x = ...) != null)`.
- `break` **al final de un bucle** (4). Se usan para buscar el primer elemento.
- **Devolver** `null` **en lugar de una colección vacía** (4). En esos métodos (`@Nullable`), `null` significa «no hay» y se distingue de «vacío».
- **Una visibilidad cambiada con** `setAccessible` (1). Es para registrar estructuras en Forge 1.16.5, que no tiene una forma pública de hacerlo.
- `if` **que se podría escribir como un solo** `return` (2). Se dejó así porque se lee mejor.
- **Un parámetro sin usar** (1): el constructor de `CimientoPiece` tiene la firma que exige Forge para cargar piezas de estructura.
- **Un campo no serializable en un** `enum` (1): `ArmasCriollas` guarda un proveedor de ítem de reparación. Los `enum` de Java no se serializan campo por campo, así que no tiene efecto.

**Falsos positivos excluidos:** `CloseResource` marca 41 usos de `Minecraft`, `World` y `MinecraftServer` porque implementan `AutoCloseable`. Son objetos del juego que el mod nunca abre ni debe cerrar, y cerrarlos rompería la partida. Por eso la regla está excluida, con su justificación, en el archivo de reglas.

## 8.2 Herramientas de Python

Las herramientas de `tools/` se revisaron con **pyflakes**: no quedan imports ni variables sin usar. Las plantillas de estructuras usan `from estructuras_base import *` a propósito: es un pequeño lenguaje para describir edificios (bloques, paredes, techos).

## 8.3 Buenas prácticas aplicadas

- **Separación en capas.** El motor del truco (`core`) no conoce Minecraft, la lógica de servidor no conoce las pantallas, y las clases de cliente están marcadas con `@OnlyIn(Dist.CLIENT)` o se cargan por `DistExecutor`. Así el servidor dedicado arranca sin código de pantalla.
- **Validación de toda entrada del cliente.** Cada paquete limita tamaños y el servidor valida asiento, turno, distancia y legalidad (ver RNF-S1 a RNF-S5).
- **Manejo de errores.** Las escenas, cinemáticas y efectos están protegidos con `try/catch`: si algo falla, se corta esa escena, se registra en el log y el juego sigue. Los errores de generación del mundo se registran sin tumbar el servidor.
- **Sin credenciales ni secretos.** El repositorio no contiene claves ni tokens, y el mod no se conecta a internet mientras se juega.
- **Pruebas automáticas e integración continua:** 33 pruebas en cada push (GitHub Actions).
- **Nombres y comentarios.** Clases y métodos con nombres descriptivos. El código de Minecraft sigue los nombres oficiales (en inglés); las partes propias del juego van en español rioplatense, como el resto del proyecto.